

Cahier des charges - Projet Fil Rouge CDA

Plateforme de Gestion RH d'Entreprise

Informations générales

Titre du projet : Plateforme de Gestion RH d'Entreprise **Type** : Application Web sécurisée **Contexte** : Projet fil rouge pour la certification CDA (RNCP Niveau 6) **Objectif** : Valider les 3 CCP du titre professionnel **Durée de développement** : 8-10 mois (tout au long de la formation)

Objectifs pédagogiques

Ce projet doit permettre de valider les compétences suivantes :

CCP1 - Développer une application sécurisée

- Configurer un environnement de développement professionnel
- Développer des interfaces utilisateur responsive et accessibles
- Développer des composants métier sécurisés
- Contribuer à la gestion de projet (versioning, documentation)

CCP2 - Concevoir et développer une application organisée en couches

- Analyser les besoins et créer des maquettes
- Définir une architecture logicielle multicouche
- Concevoir et implémenter une base de données relationnelle
- Développer des composants d'accès aux données

CCP3 - Préparer le déploiement d'une application sécurisée

- Écrire et exécuter des tests (unitaires, d'intégration)
 - Documenter le déploiement de l'application
 - Mettre en place une démarche DevOps (CI/CD)
-

Contexte métier

L'entreprise **TechCorp** (fictive) souhaite moderniser sa gestion des ressources humaines en remplaçant ses outils disparates (Excel, emails, papier) par une plateforme web centralisée et sécurisée.

Problématiques actuelles

- Processus de demande de congés chronophage et peu traçable
- Gestion du matériel informatique complexe (qui a quoi ?)
- Traitement des notes de frais long et source d'erreurs
- Manque de visibilité pour les managers et la direction RH

Solution attendue

Une application web permettant de :

- Digitaliser les processus RH
 - Centraliser les informations
 - Automatiser les workflows de validation
 - Générer des rapports et statistiques
 - Garantir la sécurité et la confidentialité des données (RGPD)
-

Acteurs du système

1. Employé

- Consulte son profil et ses informations personnelles
- Demande des congés
- Consulte ses soldes de congés
- Demande du matériel
- Soumet des notes de frais
- Consulte l'historique de ses demandes

2. Manager / Responsable d'équipe

- Toutes les actions d'un Employé
- Valide ou refuse les demandes de congés de son équipe
- Valide ou refuse les notes de frais de son équipe
- Consulte les congés de son équipe (planning)
- Reçoit des notifications pour les demandes en attente

3. RH / Administrateur RH

- Toutes les actions d'un Manager
- Valide ou refuse les demandes après validation manager
- Gère les utilisateurs (création, modification, désactivation)
- Gère le catalogue de matériel
- Attribue du matériel aux employés
- Génère des rapports (congés, notes de frais, matériel)
- Consulte les tableaux de bord statistiques

4. Administrateur Système

- Gestion complète des utilisateurs et des rôles
 - Configuration de l'application
 - Consultation des logs
 - Gestion de la sécurité
-

Fonctionnalités attendues

Module 1 : Gestion des utilisateurs et authentification

Fonctionnalités obligatoires

- ☐ **Inscription** : Formulaire avec validation (email, mot de passe fort)
- ☐ **Connexion sécurisée** : JWT ou sessions, limitation des tentatives
- ☐ **Gestion de profil** : Modification des informations personnelles
- ☐ **Rôles et permissions** : Employé, Manager, RH, Admin
- ☐ **Réinitialisation de mot de passe** : Par email (fictif ou réel)
- ☐ **Déconnexion** : Invalidation de session/token

Fonctionnalités optionnelles

- Double authentification (2FA)
- Connexion via OAuth (Google, Microsoft)
- Historique des connexions

Critères de validation

- Mots de passe hachés (BCrypt, Argon2)
 - Protection contre les attaques (brute force, injection SQL, XSS)
 - Conformité RGPD (consentement, données minimales)
-

Module 2 : Gestion des congés

Fonctionnalités obligatoires

- ☐ **Demande de congés** :
 - Sélection de dates (début, fin)
 - Type de congé (CP, RTT, maladie, sans solde)
 - Motif optionnel
 - Calcul automatique du nombre de jours
- ☐ **Validation à 2 niveaux** :
 - Niveau 1 : Manager (approuve ou refuse)
 - Niveau 2 : RH (validation finale)
- ☐ **Consultation des soldes** :
 - Solde initial
 - Congés pris
 - Congés en attente
 - Solde restant
- ☐ **Historique des demandes** :
 - Liste des demandes (en attente, validées, refusées)
 - Statut et dates de traitement

- ☐ **Planning d'équipe :**
 - Visualisation des absences de l'équipe (calendrier)
 - Détection des conflits (trop d'absences simultanées)

Fonctionnalités optionnelles

- Export du planning (PDF, iCal)
- Notifications par email/SMS
- Règles métier (ex: maximum 3 personnes absentes en même temps)
- Gestion des jours fériés

Critères de validation

- Workflow de validation respecté
 - Calcul correct des jours ouvrés
 - Impossible de demander des congés avec un solde insuffisant
 - Les managers ne voient que leur équipe
-

Module 3 : Gestion du matériel

Fonctionnalités obligatoires

- ☐ **Catalogue de matériel :**
 - Liste du matériel disponible (ordinateurs, téléphones, écrans, etc.)
 - Informations : nom, type, numéro de série, état
- ☐ **Attribution de matériel :**
 - RH attribue du matériel à un employé
 - Date d'attribution, état à la sortie
- ☐ **Retour de matériel :**
 - Enregistrement du retour
 - État à la rentrée
 - Disponibilité dans le catalogue
- ☐ **Inventaire :**
 - Liste du matériel par employé
 - Liste du matériel disponible/en service/en réparation
- ☐ **Demande de matériel :**
 - L'employé peut demander du matériel
 - Validation par la RH

Fonctionnalités optionnelles

- Historique complet du matériel (tous les utilisateurs)
- Alertes (matériel non rendu depuis > 2 ans)
- Gestion des garanties

Critères de validation

- Un matériel ne peut être attribué qu'à une seule personne à la fois
 - Traçabilité complète des mouvements
 - Statuts cohérents (disponible, attribué, en réparation, hors service)
-

Module 4 : Gestion des notes de frais

Fonctionnalités obligatoires

- ☐ **Soumission de note de frais :**
 - Date de la dépense
 - Type (transport, restauration, hébergement, autre)
 - Montant TTC
 - Upload du justificatif (PDF, image)
 - Description
- ☐ **Validation à 2 niveaux :**
 - Niveau 1 : Manager
 - Niveau 2 : RH (validation finale et traitement du remboursement)
- ☐ **Consultation des notes de frais :**
 - Liste des notes (en attente, validées, refusées, remboursées)
 - Détails et téléchargement du justificatif
- ☐ **Traitement RH :**
 - Marquer comme "remboursée"
 - Ajout d'un commentaire

Fonctionnalités optionnelles

- Calcul du montant total par mois/année
- Export comptable (CSV, Excel)
- Plafonds de dépenses par catégorie
- OCR pour extraction automatique des données du justificatif

Critères de validation

- Fichiers uploadés stockés de manière sécurisée
 - Validation des formats de fichiers (PDF, JPG, PNG)
 - Limitation de la taille des uploads (ex: 5 Mo)
-

Module 5 : Tableaux de bord et rapports

Fonctionnalités obligatoires

- ☐ **Tableau de bord Employé :**
 - Résumé des congés (solde, prochaines absences)
 - Résumé du matériel attribué
 - Notes de frais en attente de traitement
- ☐ **Tableau de bord Manager :**
 - Demandes en attente de validation (congés, notes de frais)
 - Vue d'ensemble de l'équipe (présence/absence)
- ☐ **Tableau de bord RH :**
 - Statistiques globales :
 - Nombre de demandes de congés par mois
 - Taux d'utilisation des congés
 - Total des notes de frais par catégorie
 - Inventaire du matériel
 - Graphiques (barres, camemberts, courbes)

Fonctionnalités optionnelles

- Export des rapports (PDF, Excel)
- Filtres avancés (par service, par période, par type)
- Graphiques interactifs (Chart.js, D3.js)

Critères de validation

- Données agrégées correctement
 - Performances acceptables (requêtes optimisées)
 - Graphiques clairs et lisibles
-

Module 6 : Notifications (optionnel mais recommandé)

- ☐ Notification lors d'une nouvelle demande (Manager, RH)
 - ☐ Notification de validation/refus (Employé)
 - ☐ Rappels (demandes en attente depuis > 7 jours)
 - ☐ Centre de notifications dans l'application
 - ☐ Optionnel : emails de notification
-

Contraintes techniques

Architecture obligatoire

L'application **DOIT** respecter une architecture **multicouche (n-tiers)** :

1. Frontend (Présentation) :

- Interface utilisateur responsive
- Compatible navigateurs modernes (Chrome, Firefox, Safari, Edge)
- Accessible (WCAG 2.1 niveau AA)

2. Backend (Logique métier) :

- API REST sécurisée
- Gestion de l'authentification et des autorisations
- Validation des données côté serveur

3. Base de données (Persistance) :

- Relationnelle (PostgreSQL recommandée, MySQL acceptable)
- Schéma normalisé (au moins 3NF)
- Optionnel : NoSQL pour des fonctionnalités spécifiques

Choix technologiques

Les technologies sont **LIBRES** mais doivent être **justifiées** dans le dossier professionnel.

Stack Backend (choisir 1 option)

Option 1 : Node.js (Recommandé pour débutants)

- **Framework** : Express.js ou NestJS
- **ORM** : Sequelize ou TypeORM
- **Avantages** : JavaScript full-stack, asynchrone, npm riche
- **Inconvénients** : Moins structuré par défaut

Option 2 : PHP (Recommandé pour projets métier)

- **Framework** : Symfony ou Laravel
- **ORM** : Doctrine ou Eloquent
- **Avantages** : Orienté web, frameworks matures, documentation riche
- **Inconvénients** : Moins moderne que Node.js

Option 3 : Java (Recommandé pour projets entreprise)

- **Framework** : Spring Boot
- **ORM** : Hibernate
- **Avantages** : Robuste, typé, adapté aux grosses applications
- **Inconvénients** : Verbeux, courbe d'apprentissage

Option 4 : Python (Alternative)

- **Framework** : Django ou FastAPI
- **ORM** : Django ORM ou SQLAlchemy
- **Avantages** : Syntaxe claire, polyvalent
- **Inconvénients** : Moins utilisé en web entreprise que les autres

Stack Frontend (choisir 1 option)

Option 1 : Vanilla JS (Débutant)

- HTML5 + CSS3 + JavaScript ES6+
- Pas de framework lourd
- **Avantages** : Maîtrise des fondamentaux
- **Inconvénients** : Plus de code, moins maintenable

Option 2 : Framework JS moderne (Recommandé)

- **React** : Bibliothèque, flexible, très utilisé
- **Vue.js** : Framework progressif, facile à apprendre
- **Angular** : Framework complet, structuré
- **Avantages** : Composants, état réactif, écosystème riche
- **Inconvénients** : Courbe d'apprentissage

Base de données (choisir 1 ou 2 options)

Obligatoire : SQL

- **PostgreSQL** (recommandé) : Robuste, features avancées, open-source
- **MySQL/MariaDB** : Populaire, simple, bien documenté

Optionnel : NoSQL

- **MongoDB** : Pour des fonctionnalités non relationnelles (logs, notifications)

Sécurité (OBLIGATOIRE)

L'application **DOIT** implémenter :

- ☒ **Authentification sécurisée** :
 - Hachage des mots de passe (BCrypt, Argon2)
 - Politique de mots de passe forts
 - Limitation des tentatives de connexion
- ☒ **Autorisation** :
 - Contrôle d'accès basé sur les rôles (RBAC)
 - Vérification des permissions à chaque requête
 - Principe du moindre privilège
- ☒ **Protection contre les failles** :
 - Injection SQL (requêtes préparées, ORM)
 - XSS (échappement, CSP)
 - CSRF (tokens)
 - Clickjacking (X-Frame-Options)
- ☒ **HTTPS** (en production)

- ☒ **Conformité RGPD :**
 - Consentement pour le traitement des données
 - Droit d'accès, de rectification, d'effacement
 - Pseudonymisation des données sensibles
 - Logs des accès aux données personnelles

Tests (OBLIGATOIRE)

L'application **DOIT** inclure :

- ☒ **Tests unitaires :**
 - Couverture minimale : 70%
 - Tests des fonctions métier critiques
 - Tests des validations
- ☒ **Tests d'intégration :**
 - Tests des routes API
 - Tests des accès à la base de données
- ☒ **Tests de sécurité :**
 - Tests des autorisations
 - Tentatives d'accès non autorisées
- ☐ **Tests E2E** (optionnel mais recommandé) :
 - Scénarios utilisateur complets

Outils recommandés :

- JavaScript : Jest, Mocha, Chai
- PHP : PHPUnit
- Java : JUnit, Mockito
- Python : pytest

DevOps (OBLIGATOIRE)

L'application **DOIT** être :

- ☒ **Versionnée avec Git :**
 - Repository GitHub/GitLab
 - Commits réguliers et clairs
 - Branches pour les fonctionnalités
 - Pull requests / Merge requests
- ☒ **Conteneurisée avec Docker :**
 - Dockerfile pour chaque service
 - Docker Compose pour l'orchestration

- Variables d'environnement pour la configuration
- ☒ **CI/CD :**
 - Pipeline automatique (GitHub Actions, GitLab CI)
 - Tests automatiques à chaque push
 - Déploiement automatique (staging)

Documentation (OBLIGATOIRE)

- ☒ **README.md :**
 - Description du projet
 - Prérequis
 - Installation
 - Utilisation
 - Technologies utilisées
 - ☒ **Documentation de l'API :**
 - Swagger / OpenAPI (recommandé)
 - Endpoints, méthodes, paramètres, réponses
 - ☒ **Documentation du code :**
 - Commentaires pour les parties complexes
 - JSDoc, PHPDoc, Javadoc selon le langage
 - ☒ **Guide de déploiement :**
 - Étapes de déploiement
 - Configuration de l'environnement de production
 - Gestion des secrets
-

Conception attendue

Maquettes (OBLIGATOIRE)

Créer des maquettes pour les écrans principaux :

- Page de connexion
- Tableau de bord (Employé, Manager, RH)
- Formulaire de demande de congés
- Liste des demandes (avec filtres)
- Détail d'une demande
- Gestion du matériel
- Planning d'équipe

Outils recommandés : Figma, Adobe XD, Sketch, Penpot (open-source)

Diagrammes UML (OBLIGATOIRE - choisir selon préférence)

Option 1 : UML

- Diagramme de cas d'utilisation (use cases)
- Diagramme de classes
- Diagramme de séquence (pour les workflows principaux)

Option 2 : MERISE

- Modèle Conceptuel de Données (MCD)
- Modèle Logique de Données (MLD)
- Modèle Physique de Données (MPD)

Outils recommandés :

- UML : PlantUML, Draw.io, Lucidchart
- MERISE : JMerise, AnalyseSI, Looping

Schéma d'architecture (OBLIGATOIRE)

Schéma montrant :

- Frontend (navigateur)
- Backend (API)
- Base de données
- Services externes (email, stockage)
- Flux de données

Dictionnaire de données (OBLIGATOIRE)

Tableau décrivant chaque entité et ses attributs :

- Nom de la table
- Nom de la colonne
- Type
- Contraintes
- Description

Exigences UI/UX

Design

- Interface **responsive** (mobile, tablette, desktop)
- **Accessibilité** WCAG 2.1 niveau AA :
 - Contraste suffisant
 - Navigation au clavier
 - Textes alternatifs pour les images
 - Labels pour les formulaires
- **Charte graphique cohérente** :

- Palette de couleurs définie
- Typographie lisible
- Espacements réguliers
- **Feedback utilisateur :**
 - Messages de succès/erreur clairs
 - Loaders pendant les chargements
 - Confirmation avant les actions destructrices

Expérience utilisateur

- Navigation **intuitive**
 - Formulaires **simples** avec validation côté client et serveur
 - Messages d'erreur **explicites**
 - Temps de chargement **optimisés** (< 3 secondes)
-

Livrables attendus

1. Code source

- Repository Git (GitHub/GitLab) avec :
 - Code complet et commenté
 - Fichiers de configuration (.env.example)
 - Scripts de déploiement
 - Tests

2. Documentation

- README.md complet
- Documentation de l'API (Swagger)
- Guide de déploiement
- Cahier de recette (scénarios de tests)

3. Dossier de conception

- Cahier des charges (ce document)
- Maquettes UI/UX
- Diagrammes UML ou MERISE
- Schéma d'architecture
- Dictionnaire de données

4. Dossier professionnel

- Description du projet et contexte
- Conception (maquettes, diagrammes)
- Extraits de code commentés
- Tests et qualité
- Déploiement

- Veille technologique et justification des choix

5. Support de présentation

- Slides pour la soutenance (15-20 slides max)
 - Démo vidéo (optionnel mais recommandé)
-

Planning indicatif

Phase 1 : Cadrage et conception (Semaines 3-6)

- Lecture du cahier des charges
- Choix des technologies (justifiés)
- Création des maquettes
- Modélisation de la base de données (MCD/UML)
- Schéma d'architecture
- Initialisation du repository Git

Phase 2 : Développement du MVP (Semaines 7-15)

- Configuration de l'environnement
- Mise en place de la base de données
- Développement du backend (API)
 - Authentification
 - Gestion des utilisateurs
 - Module congés (fonctionnalités de base)
- Développement du frontend (pages principales)
- Tests unitaires

Phase 3 : Développement des modules complémentaires (Semaines 16-25)

- Module matériel
- Module notes de frais
- Tableaux de bord
- Notifications
- Tests d'intégration

Phase 4 : Sécurisation et optimisation (Semaines 26-30)

- Audit de sécurité (tests OWASP)
- Optimisation des performances
- Corrections de bugs
- Tests de charge (optionnel)

Phase 5 : Déploiement et finalisation (Semaines 31-35)

- Dockerisation
- Mise en place CI/CD
- Déploiement en staging

- Documentation complète
 - Préparation du dossier professionnel
 - Préparation de la soutenance
-

Critères de réussite

Fonctionnalités (40%)

- ☐ Authentification sécurisée fonctionnelle
- ☐ Les 4 rôles sont implémentés avec les bonnes permissions
- ☐ Module congés complet (demande, validation, planning)
- ☐ Module matériel fonctionnel
- ☐ Module notes de frais fonctionnel
- ☐ Tableaux de bord avec statistiques

Architecture et code (30%)

- ☐ Architecture multicouche respectée
- ☐ Code propre et organisé (principes SOLID)
- ☐ API REST respectant les standards
- ☐ Base de données normalisée
- ☐ Pas de failles de sécurité critiques

Tests et qualité (15%)

- ☐ Couverture de tests > 70%
- ☐ Tests unitaires et d'intégration
- ☐ Tests de sécurité (tentatives d'accès non autorisées)
- ☐ Application stable, pas de bugs bloquants

Déploiement (10%)

- ☐ Application déployée et accessible
- ☐ Docker configuré
- ☐ CI/CD fonctionnel
- ☐ Documentation du déploiement claire

Documentation (5%)

- ☐ README complet
 - ☐ Documentation de l'API (Swagger)
 - ☐ Diagrammes et maquettes de qualité
 - ☐ Dossier professionnel complet
-

Erreurs à éviter

-Commencer à coder sans phase de conception -Négliger la sécurité ("je le ferai plus tard") -Ne pas tester son code -Choisir des technologies non enseignées sans justification -Copier-coller du code sans

le comprendre (plagiat détecté) -Ignorer le versioning Git -Partir sur un projet trop ambitieux -Attendre la dernière minute pour la documentation -Ne pas faire de sauvegardes régulières

Conseils

Démarrer simplement

- Commencez par un MVP (Minimum Viable Product)
- Ajoutez les fonctionnalités une par une
- Testez au fur et à mesure

Organiser son code

- Respectez les conventions de nommage
- Utilisez des dossiers clairs (routes, controllers, models, services)
- Commentez les parties complexes

Gérer son temps

- Fixez-vous des deadlines intermédiaires
- Avancez régulièrement (1-2h par jour mieux que 10h une fois par mois)
- N'hésitez pas à demander de l'aide aux formateurs

Préparer la soutenance

- Préparez une démo fluide (environnement de démo stable)
 - Ayez un plan B si la démo plante (vidéo, screenshots)
 - Entraînez-vous à présenter votre projet
 - Anticipez les questions du jury
-

Ressources

Documentation officielle

- [MDN Web Docs](#) (HTML, CSS, JS)
- [Node.js Documentation](#)
- [Symfony Documentation](#)
- [Laravel Documentation](#)
- [Spring Boot Documentation](#)
- [PostgreSQL Documentation](#)

Sécurité

- [OWASP Top 10](#)
- [ANSSI - Recommandations](#)
- [RGPD - CNIL](#)

Outils

- [Figma](#) (maquettes)
 - [PlantUML](#) (diagrammes UML)
 - [Swagger](#) (documentation API)
 - [Docker](#)
 - [GitHub Actions](#)
-

Contact et support

Pour toute question sur le projet fil rouge :

- **Formateur référent** : Mounir
 - **Plateforme** : Microsoft Teams (canal de la promo)
 - **Email** : [À compléter]
 - **Heures de questions dédiées** : [À définir]
-

Évolutions possibles (bonus)

Si vous terminez le projet de base, voici des évolutions possibles pour aller plus loin :

- **Messagerie interne** entre employés
 - **Gestion des entretiens annuels**
 - **Module de formation** (demande, suivi)
 - **Gestion des absences imprévues** (maladie avec arrêt)
 - **Application mobile** (React Native, Flutter)
 - **Notifications push**
 - **Internationalisation** (i18n - anglais, français)
 - **Mode offline** (PWA)
 - **Export avancé** (Excel, PDF personnalisés)
 - **Intégration Slack/Teams** pour les notifications
-

Dernière mise à jour : Septembre 2026 **Version** : 1.0